Ex 1.

G0. calculator mécaniques

G1. " " " électronique

G2. " " à transistors

G3. " " à circuit intégrés

G4. VLSI

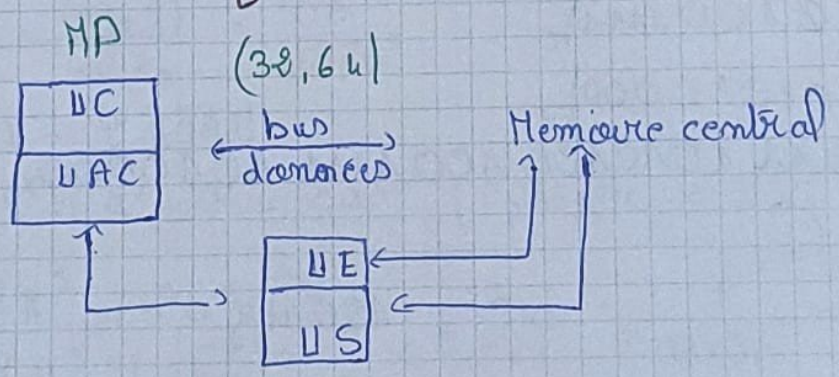
2) La machine Von Neumann.

- Mémoire centrale
- Unité de commande
- Unité Arithmetique et logique
- Unité d'entrée
- Unité de Sortie

3)

Ordinateur { Hardware. materiel ⇒ RAM. disque dur
Software logiciel Apps. sys d'exploitations OS

4)



5) RAM. Mémoire de traitement

ROM. mémoire de stockage

Ex 2.

CPU. central processing Unite

UAL. Unite Arithmetique et logique

$(432)_{10} = (110110000)_2$

$X = (-1)^0 \times 2^8 \times 1,1011$

$E + 127 = (135)_{16} = (10000111)_2$

4 3 D 8 0 0 0 0

$\underbrace{0}\,\underbrace{1000\,0111}\,\underbrace{1011\,1000}\,\underbrace{0000\,0000}\,\underbrace{0000\,0000}\,\underbrace{0000\,0000}$

$(432)_{16} = 43D80000$ IEEE

10) $(415A0000)$ IEEE

0 | 100 0001 0 | 101 1010 0000 0000 0000 0000

$S = 0$

$E + 127 = (100\,00010)_2 = 128 + 2 = 130$

$E = 130 - 127 = 3$

$f = 0,101101$

$X = (-1)^0 \times 2^3 \times 1,101101$

$X = 1101,101$

$= 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-3}$

$= 8 + 4 + 1 + 0,5 + 0,125$

$= (13,625)_{10}$

11) J'AI TROUVE !

12) $(100\,1010)_2 = (4A)_{16} = J$

RAM, Random Access Memory

ROM, Read only Memory

USB, Universal serie bus

Ex3,

1) $|-34| = 34$

$(34)_{10} = (100010)_2$

$(34)_{10} = (00100010)_2$

$C_1(-34) = (11011101)_2$

$C_2(-34) = C_1(-34) + 1 = 11011110 \Rightarrow C$

2) $C_2(x) = 10010011$

$C_1(x) = 10010010$

$|x| = (01101101)_2$

$= 1\times2^6 + 1\times2^5 + 1\times2^3 + 1\times2^2 + 1\times2^0$

$= 64 + 32 + 8 + 4 + 1$

$= 109$

$x = -109$

3) $S = (53)_{16} = (01010011)_2$

$T = (54)_{16} = (01010100)_2$

$O = (4F)_{16} = (01001111)_2$

$P = (50)_{16} = (01010000)_2$

STOP = 10100111010100 1001111 10100000 B

4) 32 bits ©

5) $f = \frac{1}{T} = \frac{1}{10\times10^{-3}} = 10^2 = 100\ Hz$

Algo Ex 1

Variables A, B, entier

Debut

ecrire("Donner deux entiers")

lire (A, B)

ecrire ("la somme est", A+B)

ecrire ("le produit est", A*B)

ecrire ("la moyenne est", (A+B)/2)

Fin

Ex 2.

Algo

Variables R_1, R_2, R_3, RS, RP, double

debut

Ecrire ("Donnez les valeurs des 3 resistances")

lire (R_1, R_2, R_3)

RS ← $R_1 + R_2 + R_3$

RP ← $(R_1 * R_2 * R_3) / (R_1 * R_2 + R_1 * R_3 + R_2 * R_3)$

Ecrire ("Si en serie la resistance equivalente est", RS)

Ecrire ("Si en para la resitance equivalente est", RP)

Fin

Ex 3.

TD Algo 1

Ex 4

1) $(0,98)_{10} = (0,11111010)_2$

$(0,25)_{10} = (0,01)_2$

$0,25 \times 2 = 0,5$

$0,5 \times 2 = 1$

$(0,01)_2 = 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = \frac{1}{2^2} = \frac{1}{4}$

$0,98 \times 2 = 1,96$

$0,96 \times 2 = 1,92$

$0,92 \times 2 = 1,84$

$0,84 \times 2 = 1,68$

$0,68 \times 2 = 1,36$

$0,36 \times 2 = 0,72$

$0,72 \times 2 = 1,44$

$0,44 \times 2 = 0,88$

$0,88 \times 2 = 1,76$

2) $(38)_{16} = (00111000)_2$

$(A14)_{16} = (1010\ 0001\ 0100)_2$

$(FB17)_{16} = (1111\ 1011\ 0001\ 0111)_2$

3) $(1110)_{16} = (E)_{16}$

$(10111)_{16} = (17)_{16}$

$(10100110)_2 =$

$(110110)_2 = (36)_{16}$

$(110111)_2 = (37)_{16}$

4) $1A_{16} = 1 \times 16^1 + 10 \times 16^0 = 16 + 10 = (26)_{10}$

5) $(33)_{10} = (21)_{16}$

$$\begin{array}{r|l} 33 & 16 \\ 2 & 16 \\ 0 & \end{array} \quad \begin{array}{l} 1 \\ 2 \uparrow \end{array}$$

6) $(12)_8 = 1 \times 8 + 2 \times 8^0 = 8 + 2 = (10)_{10}$

7) $(110011,11)_2 = 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2$

$= 32 + 16 + 2 + 1 + 0,5 + 0,25$

$= (51,75)_{10}$

8) $(11100) = X$

$X = (00011100)$

$C_1 = 11100011$

$C_2 = 11100100$

TD Algo

Serie 5,

Ex 1,

1) Préprocessing
2) fct principale main() { },
3) Déclaration des variables

   U[50] V[50], N, i, ;
4) taille et remplissage
5) Calcul de Produit Scalaire
6) Affichage du PS

```
# include <stdio.h>
main ( ) {
// Declaration
int U[50];
int V[50];
int i, N, PS;
// taille et saisie
printf ("Donner la taille des deux vecteurs");
scanf ("%d", &N);
for (i=0; i<N, i++) {
    printf ("Donner un élément du vecteur U");
    scanf ("%d", &U[i]);
    printf ("Donner élement du vecteur V");
    scanf ("%d", &V[i]); }
// calcul du Produit scalaire,
PS=0;
for(i=0; i<N; i++)
```

```
PS = PS + U[i] * V[i];
printf ("le produit scalaire est %d", PS);
return 0;
}
```

Ex2.

```
# include <stdio.h>
main () {
// Déclaration
int A [50][50];
float B [50][50];
int i, j, L, C;
float x;
// taille et saisie
printf ("Donner le reel ");
scanf ("%f", &x);
printf ("Donner le nombre de lignes et de colonnes");
scanf ("%d %d", &L, &C);
// Remplissage
  for (i=0; i<L; i++)
      for (j=0; j<C; j++) {
      printf ("entrer un element");
      scanf ("%d", &A[i][j]); }
// Calcul produit Matrice x reel
for (i=0, i<L; i++)
    for (j=0; j<C; j++)
B [i][j] = A[i][j] * x;
// Affichage
```

```
Som ← Som + 1/i
fin pour
ecrire (" la somme est", Som)
fin si
Fin
```

Ex 4,

```
Algo - Tab
Variables   i, N, entier
            Som, Moy, Prod : double
    Tableau T[60] : entier
Debut
   ecrire(" Donner le nombre des éléments")
   lire (N)
pour i allant de 0 jusqu'à N-1 avec pas 1
   ecrire ("entrer un element")
   lire (T[i])
  fin pour
  Som ← 0
  Prod ← 1
Pour i allant de 0 à N-1 avec pas 1
   Som ← Som + T[i]
   Prod ← Prod * T[i]
fin pour
   Moy ← Som/N
  ecrire("la somme est" Som)
  ecrire ("le produit est", Prod)
  ecrire(" la moyenne est", moy)
Fin
```

```
for (i=0, i<L ; i++)
    for (j=0 ; j<C ; j++)
  printf("%f", B[i][j]);
return 0;
}
```

Ex3

```
# include <stdio.h>
main ( ) {
// Déclaration
int T[50]
int TPOS[50];
int TNEg[50];
int i, N , POS, Neg;
// taille et saisie
printf ("Donner la taille du tableau");
scanf ("%d", & N);
// Remplissage
for (i=0 ; i < N ; i++) {
    printf ("Donner un élément");
    scanf ("%d", &T[i]); }
// Affichage
for(i=0; i<N, i++)
printf ("%d" ; T[i]);
// Traitement
POS = 0;
Neg = 0;
for (i=0 ; i < N ; i++)
if (T[i] > 0) {
```

TD Algorithme

S N°1

Ex 1

Mot reservé : c'est un mot clé qui a une signification spéciale dans un langage de programmation

Variable : c'est un emplacement mémoire attaché à un nom dont le contenu peut se changer au cours du programme.

Algorithme : c'est une suite finie d'instruction pour resoudre un problème

2) Énoncé du Problème

Cahier des charges } Spécification

Algorithme } Analyse

programme source } Traduction

programme executable } Compilation

Version finale } test maintenance

3) Algo → pseudo-code, → Organigramme

4) A $= (65)_{10}$
a $= (97)_{10}$ } code ASCII

"A" < "a"

Ex 2

1 → C

2 → B

3 → B

4 → A, D

5 → A, C

Ex 3

$$\begin{array}{r} 00101001_2 \\ +\ 11001010 \\ \hline 11110011 \end{array} \qquad \left.\begin{array}{r} 10101011 \\ 11001010 \\ \hline 101110101 \end{array}\right\}$$

Over flow = Depassant memoire

2) Algo Echange

Variables A, B, entier

debut

ecrire ("Donner deux entiers A et B")

lire (A, B)

ecrire ("la valeur de A est", B)

ecrire ("la valeur de B est", A)

Fin

3) Algo - Distance

Variables X_A, X_B, Y_A, Y_B : entier

dist : double

debut

ecrire ("Donnez les coordonnées du 1er point A")

lire (X_A, Y_A)

ecrire ("Donnez les coordonnées du 2ème point B")

lire (X_B, Y_B)

Dist ← $((X_A - X_B)^2 + (Y_A - Y_B)^2))^{(1/2)}$

ecrire ("la distance est", Dist)

fin

S. N°3.

Ex 1

```
TPOS [POS] = T[i];
 POS++; }
 if (T[i]<0) {
 TNeg [Neg] = T[i];
 Neg++; }
//Affichage
 for(i=0; i<Pos, i++)
   printf("%d", TPOS[i]);
 for(i=0; i<Neg; i++)
   printf("%d" Neg[i]);
 return 0;
}
```

TD Algo

Serie 3

Ex 3,

```
Algo TTC
Variable PNet, TVA, entier
         TTC : réel
Debut
   ecrire ("Donner le prix net")
   lire (PNet)
   ecrire ("Donner le TVA")
   lire (TVA
   PTT ← (réel) PNet + PNet * (TVA/100)
   ecrire ("le PTTC est", PTTC)
fin
```

Serie 4,

Ex 4

```
Algo -signe-Produit
Variable A, B : entier
Debut
   ecrire ("Donner les deux entiers")
   lire (A, B)
  Si (A=0 ou B=0) Alors
   ecrire ("le produit est nul")
   Sinon Si ((A>0 et B>0) ou (A<0 et B<0)) Alors
   ecrire ("le produit est positif")
   Sinon
   ecrire ("le produit est negatif")
   fin si
fin fin si
```

Ex 2/

```
Algo - factorielle
Variable   i, N , entier
           fact , double
Début
   ecrire (" Donner un entier")
   lire (N)
 Si (N=0) Alors
  ecrire (" la factorielle est égale à 1")
 Sinon
   fact ← 1
  pour i allant de 1 à N avec pas de 1
   fact ← fact * i
   fin pour
  ecrire (" le factorielle est ", fact)
  fin si
Fin
```

Ex 3/

```
Algo - Somme
Variable i, N , entier
         Som , double
Début
   ecrire (" Donner un entier")
   lire (N)
 Si (N=0) alors
  ecrire (" erreur")
Sinon
  Som ← 0
 pour i allant de 1 à N avec pas de 1
```